

Intrusion Detection in IoT Ecosystems Using Classifiers

Colin Moller

Northeastern University

DADS 7275: Machine Learning and Data Analytics

Professor Xuemin Jin

August 9, 2026

Abstract

This project develops and evaluates a supervised machine learning framework for intrusion detection on the RT-IoT2022 dataset (123,117 network flows, 83 features, 12 traffic types), covering two tasks: binary normal versus attack detection and multi-class attack attribution. Four classifiers (Logistic Regression, Gaussian Naive Bayes, Random Forest, and K-Nearest Neighbors) were compared. Exploratory analysis revealed that the dataset's generated attack traffic is heavily replicated: the 123,117 rows reduce to roughly 18,954 distinct flow groups, with the majority class replicated about 413 times over. Every model was therefore evaluated under two different splitting strategies. First, a conventional stratified random split and second, a flow-group split that keeps near-identical flows on one side of the train/test boundary. Binary detection proved insensitive to that choice: Random Forest achieved 0.9995 recall or better and a 0.9996 ROC-AUC under both, exceeding the project's 0.95 recall target. However, Multi-class attribution was sensitive to it. Random Forest reached a macro-F1 score of 0.9862 under the flow-group split and classified nine of twelve classes at a recall of 0.99 or better, but recall on the sparsest class fell sharply relative to the random split. Two classes of near-identical rarity moved in opposite directions under that split, which suggests the rare-class numbers depend mostly on which handful of flows ended up in the test set. Getting a trustworthy number for the rare attacks is the main open problem of this project.

1. Introduction

The expansion and maturation of Internet of Things (IoT) devices have opened up a new world of possibilities across various domains. From smart homes to factories and even smart cities, the expansion has made connection, sensing, and intelligence ubiquitous. However, this utility has come with a significant downside: it has created a whole new realm for cyber attacks.

Moreover, a confluence of factors makes hardening defenses difficult. Endpoint devices are usually simple (such as sensors) and computationally constrained, making security software difficult if not impossible, on that end. As expected, these devices are vulnerable to unauthorized access, network port scanning, DDoS, and brute-force attacks. A real need has arisen to use machine learning to monitor and classify normal vs. adversarial use in these ever more complex networks.

The decision this analysis supports is a triage one: given a flow observed at a gateway, should it be allowed through or raised as an alert, and if malicious, what kind of attack is it? The first question determines whether an operator is interrupted; the second determines what they do next. This project therefore develops and evaluates a supervised framework capable of:

- Binary Classification: Accurately flag incoming network traffic flows as either Normal (Benign) or an Attack (Malicious).
- Multi-Class Classification: Categorize malicious traffic into specific attack types.

Section 2 describes the dataset, its exploratory analysis, and the methodology, including a data quality finding that shaped the entire evaluation design. Section 3 presents binary and multi-class results, Section 4 discusses their interpretation and limitations, and Section 5 concludes with recommendations for further work.

2. Dataset and Methodology

2.1 Dataset Description

The project utilizes the RT-IoT2022 dataset, retrieved from the UCI Machine Learning Repository (Sharmila & Nagapadma, 2023). This dataset was generated using real-time IoT infrastructure, which incorporates physical edge devices alongside simulated adversary attacks.

The unit of observation is a single bidirectional network flow. There are 123,117 flow records described by 83 feature columns plus the Attack_type label, and no missing values anywhere in the dataset.

- Normal Patterns: Thing_Speak (8,108), MQTT_Publish (4,146), Wipro_bulb (253).
- Attack Patterns: DOS_SYN_Hping (94,659), ARP_poisoning (7,750), NMAP UDP/XMAS/TCP/OS/FIN scans (7,630 combined), DDOS_Slowloris (534), Metasploit_Brute_Force_SSH (37).

The predictors fall into three groups: connection metadata (proto, service, id.orig_p, id.resp_p), flow dynamics (flow_duration, fwd_pkts_per_sec, bwd_pkts_per_sec, and inter-arrival times such as fwd_iat.max), and payload and flag statistics (total payload bytes, window sizes such as fwd_init_window_size, and TCP control flag counts such as flow_SYN_flag_count and flow_FIN_flag_count). The target is Attack_type, from which both the binary and multi-class labels are derived.

2.2 Exploratory Data Analysis

The dataset is severely imbalanced. For binary classification, the dataset has 110,610 attack flows and 12,507 normal flows, about a 90/10 split of attack versus normal. Digging deeper into the data reveals that the classes of attack themselves are even more imbalanced. We have the majority class ‘DOS_SYN_Hping’ dominating the data set with 94,659 rows or about 77% of the data. On the other hand, the rarest classes have less than 40 rows! ‘Metasploit_Brute_Force_SSH’ has 37 and ‘NMAP_FIN_SCAN’ 28 rows (Figure 1). These issues will certainly need to be dealt with for a successful model.

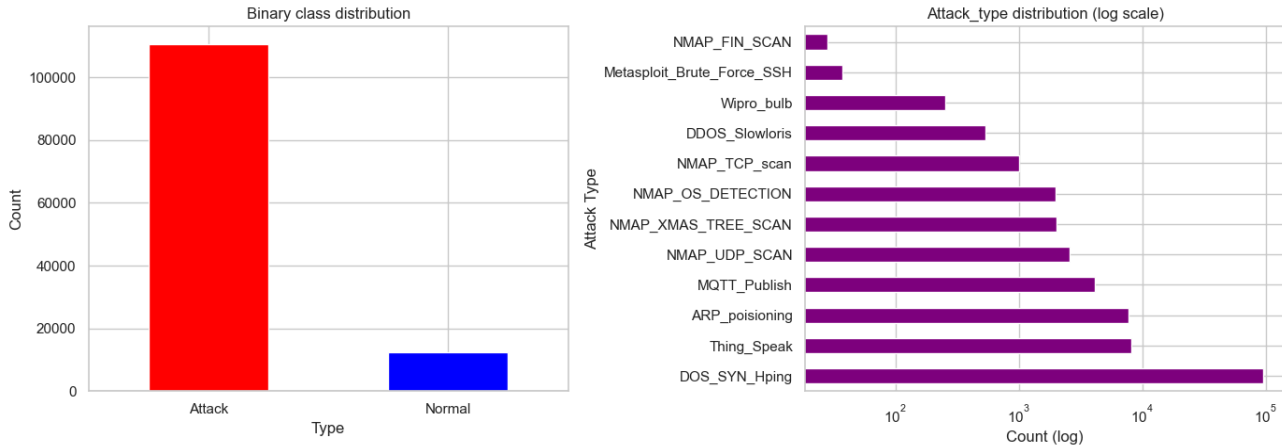


Figure 1. Binary class balance (left) and attack type distribution (right)

Running a correlation analysis found 52 feature pairs with an r value greater than 95%, with a few demonstrating near-perfect correlation. Figure 2 demonstrates the amount of redundant features in the dataset. Several pairs are identical: `flow_duration`, `fwd_iat.tot` and `flow_iat.tot` are the same quantity under three names, as are `fwd_pkts_per_sec`, `bwd_pkts_per_sec` and `flow_pkts_per_sec`. This is promising as PCA will be able to eliminate a large amount of these and streamline the dataset.

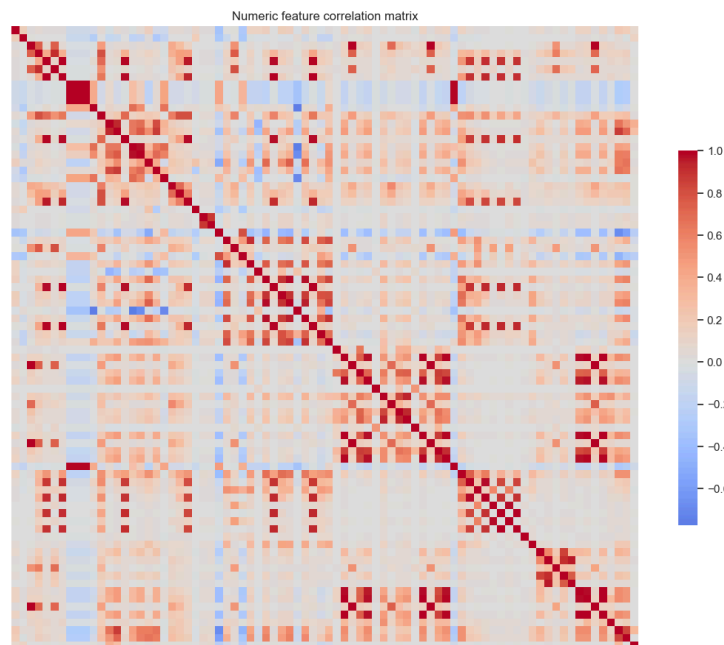


Figure 2. Correlation heat map

2.3 Data Quality and Known Challenges

Beyond the imbalance, 5,195 rows are exact duplicates of another row, concentrated in DOS_SYN_Hping (4,570) and Thing_Speak (454). That prompted a closer look because RT-IoT2022's attack traffic was synthetically produced, and it is not unreasonable to assume the tools used created flows that are near-identical. If that is the case, the row count and the amount of actual independent information in the dataset could be very different.

To measure it, I grouped flows by similarity rather than exact equality. Each numeric column was replaced by its rank-percentile and cut into 50 bins; two flows join the same “flow group” only if all 83 features land in the same bin and their categorical values match. Under that definition, the 123,117 rows collapse into 18,954 distinct flow groups. This is roughly 6.5 near-copies each. The result holds across bin sizes, where we have 15,893 groups at 20 bins and 22,814 at 200, so it is not arbitrary.

Replication varies a lot by class, and it lines up with how each class was collected (Table 1). Generated attack traffic is replicated heavily. For example, DOS_SYN_Hping's 94,659 rows are only 229 distinct flow groups, or about 413 near-copies each. Meanwhile, genuinely captured device traffic is barely replicated at all, with MQTT_Publish at 1.0 rows per group.

Class	Rows	Flow groups	Rows per group
DOS_SYN_Hping	94,659	229	413.4
NMAP_UDP_SCAN	2,590	214	12.1
NMAP_XMAS_TREE_SCAN	2,010	226	8.9
NMAP_OS_DETECTION	2,000	233	8.6
NMAP_FIN_SCAN	28	14	2.0
ARP_poisoning	7,750	5,166	1.5
DDOS_Slowloris	534	349	1.5
NMAP_TCP_scan	1,002	671	1.5
Wipro_bulb	253	216	1.2
Metasploit_Brute_Force_SSH	37	35	1.1

Class	Rows	Flow groups	Rows per group
Thing_Speak	8,108	7,483	1.1
MQTT_Publish	4,146	4,141	1.0

Table 1. Rows versus distinct flow groups by class (50-bin grouping).

This has become the central challenge of the dataset. A stratified random split treats those 413 near-copies as 413 independent observations and scatters them across train and test, so a model can score well on a test flow it has effectively already been trained on. This is a clear form of data leakage, a well-documented source of overoptimistic results in applied machine learning (Kapoor & Narayanan, 2023). Rather than just assume this either does or does not matter, I built two splits and reported both.

2.4 Data Preparation

The steps taken to get the data prepared, in this order:

1. One-hot encoding the categorical proto and service fields, and dropping the raw label columns from the feature matrix. Encoding expands the matrix to 94 features.
2. Dropping the 5,195 exact duplicate rows, leaving 117,922 flows.
3. Building the train/test splits, before any transform is fitted.
4. Standardizing numeric features inside a scikit-learn pipeline (Pedregosa et al., 2011), so the scaler is fit on training rows only and the test set is passed through transform.

If we standardized before splitting, this would let the test set's mean and variance leak into the training representation. Therefore, every transform here is a pipeline stage rather than a standalone step, which makes that error structurally impossible rather than merely avoided. Stratification to preserve the class proportions and class weighting is applied during training so the smaller attack classes aren't drowned out by the larger.

Two splits were built, both 75/25. The first is a conventional stratified random split, giving 88,441 training and 29,481 test flows. The second split stratifies over the flow groups from Section 2.3 using StratifiedGroupKFold, which keeps every class represented in the test set while guaranteeing that no group of near-identical flows is divided across the boundary; it gives 84,454 training and 33,468 test flows. To check what those two splits actually produce, I measured the distance from every test flow to its nearest training flow in a standardized PCA space fit on the training rows alone (Table 2, Figure 3).

Split	Test flow identical to a training flow	Within 0.01 of one	Median gap	Flow groups on both sides
Random	0.02%	85.98%	0.00016	1,021
Flow-group	0.00%	20.28%	0.02522	0

Table 2. Test-set novelty under the two splitting strategies.



Figure 3. Distance from each test flow to its nearest training flow, under both splitting strategies.

Under the random split, 1,021 flow groups end up on both sides of the boundary, and 86% of test flows sit essentially on top of a training flow. The flow-group split shares no groups by construction and moves the median gap by a factor of roughly 150. Though both splits are

used, I think it is important to acknowledge that the flow-group figure is the one most likely to accurately reflect model performance on genuinely unseen traffic.

2.5 Dimensionality Reduction

PCA fit to the training features reveals that 30 principal components explain more than 95% of the dataset's variance (Figure 4). This directly quantifies the redundancy seen in EDA. For Logistic Regression, Naive Bayes and Random Forest, this PCA is exploratory only, and those models are fit on the full 94-column feature set. For KNN, it is a stage of the pipeline.

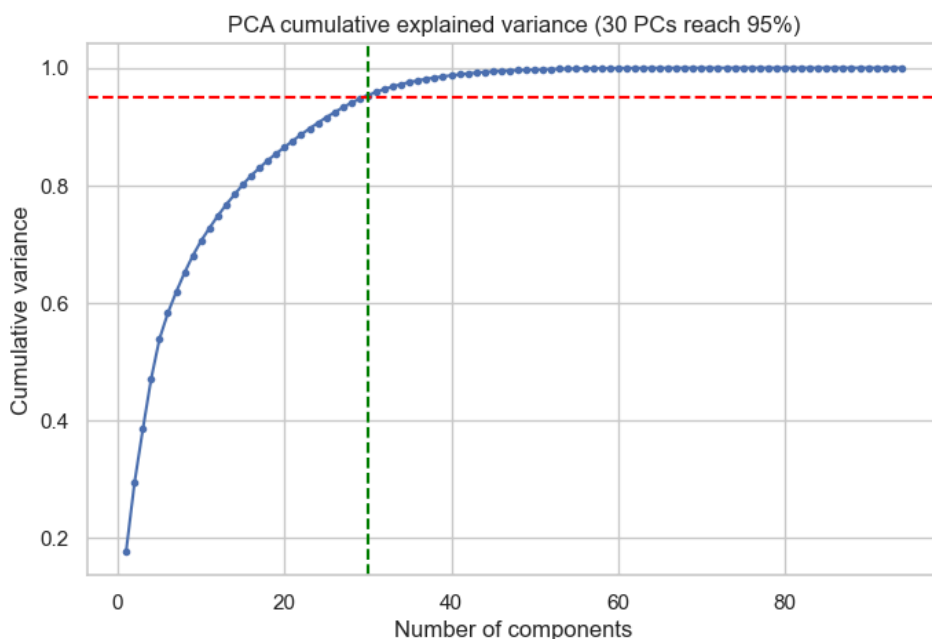


Figure 4. PCA cumulative explained variance, fit on the flow-group training set.

For the multi-class task, a t-SNE projection of a 2,000-row subsample (Figure 5) shows that most attack and traffic types form well-separated clusters, which is promising for our ability to predict them successfully. The tightness of several of these clusters is evidence of the replication talked about earlier.

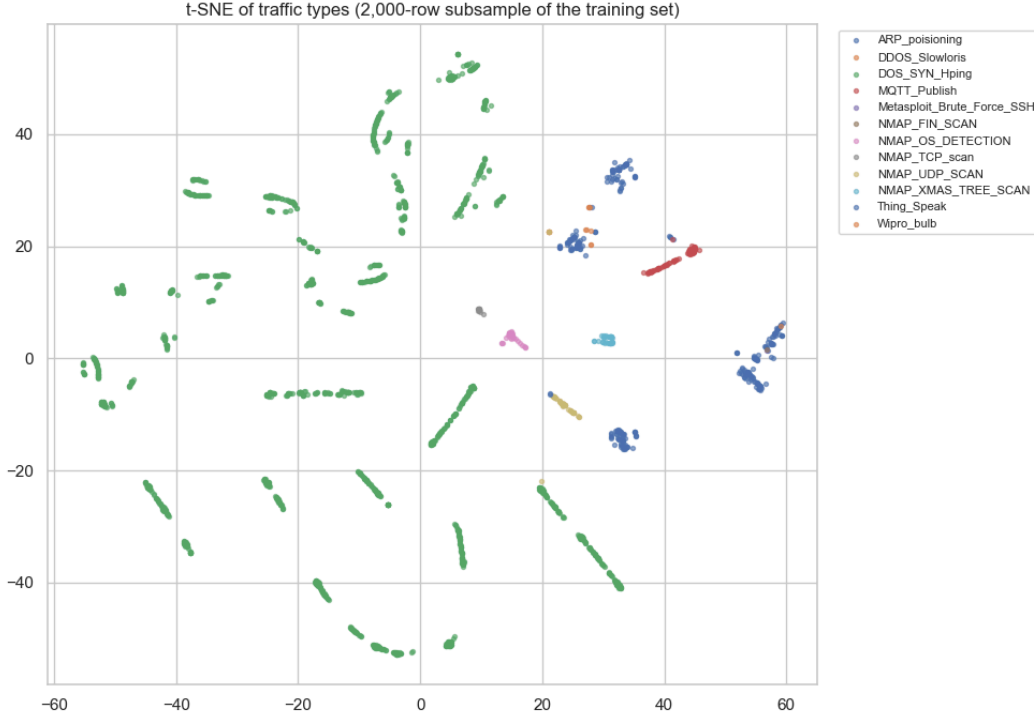


Figure 5. t-SNE projection of a 2,000-row stratified subsample of the training set.

2.6 Problem Formulation and Metrics

Supervised classification is used for both binary and multi-class predictions. For the binary task, the target y is Normal or Attack, with Attack as the positive class, and the network flow features are the input vector x . For the multi-class task, y is one of the 12 traffic types. Given both the class imbalances and the cybersecurity cost of missing an attack, the primary binary target is recall on attacks greater than 95% at a minimum, with F1 and ROC-AUC giving a more comprehensive picture.

For the multi-class task, the primary metric is macro-F1, supported by macro-recall and a balanced accuracy. Plain accuracy is reported only to show how misleading it is here. Since DOS_SYN_Hping is 77% of the rows, a model predicting that class for every flow would score 0.77 while detecting absolutely nothing. Macro-averaged metrics however, weigh the 28-row

NMAP_FIN_SCAN class the same as the 94,659-row DOS_SYN_Hping class, which is what an intrusion detector needs because a rare attack type is just as important to catch as a common one.

2.7 Models

Four algorithms were trained, with class weighting where supported, and compared for both classification tasks. Logistic Regression and Naive Bayes were chosen for their simplicity and as baselines for comparison. Random Forest was selected for its resilience to outliers, ability to handle mixed feature types, and overall strength on tabular data. K-Nearest Neighbors (KNN) chosen because it makes no assumptions about the underlying shape of the data. Its single assumption is that class membership is locally constant under the chosen distance metric, thus making it distinct and valuable for comparison in juxtaposition to the other three.

Since the chosen KNN metric is effectively the model itself, the space it operates in must be chosen deliberately. We must avoid the highly correlated features and the distance concentration of high dimension data. The model is therefore run as StandardScaler → PCA (95% variance, 30 components) → KNN and refit inside each split.

2.8 Validation Strategy

Models were fit on the training portion of each split and evaluated once on the corresponding held-out 25% test portion. Furthermore, because two splitting strategies were utilized, the comparison doubles as an interesting sensitivity analysis. It can be said that a result that holds under both splits does not depend on how the near-duplicate flows were partitioned.

3. Results and Analysis

3.1 Binary Classification

The binary classification models all exceeded expectations. All four models beat the 95% recall target that was set out and had nearly perfect F1 and ROC-AUC scores (Table 3). This speaks to attack vs. normal being readily separable in the feature space.

Model	Recall (Random)	Recall (Flow-group)	Precision (Flow-group)	F1 (Flow-group)	ROC-AUC (Flow-group)
Random Forest	0.9995	0.9997	0.9991	0.9994	0.9996
KNN (PCA)	0.9978	0.9991	0.9980	0.9985	0.9974
Naive Bayes	0.9745	0.9800	0.9683	0.9742	0.8554
Logistic Regression	0.9714	0.9760	0.9990	0.9874	0.9963

Table 3. Binary classification results under both splits. Attack is the positive class.

Although all models perform well, it is obvious that random forest is the superior choice, with the highest and near perfect score across every metric. Though the differences among models may seem small, in the realm of cybersecurity, missing even one attack may be extremely consequential. On the flow-group test set, Random Forest missed 9 attacks out of 30,469 and raised 28 false alarms out of 2,999 normal flows. The ROC curves shown in Figure 6 show the same order, with Random Forest dominating and Naive Bayes being visibly the weakest.

The more interesting result is what happens between different splitting strategies. In fact, binary detection turns out to be almost entirely insensitive to it. Random Forest scores 0.9995 and 0.9997 respectively, and the other three models all scored marginally higher on the harder flow-group split. The one exception is Naive Bayes, whose ROC-AUC falls from 0.9779 to 0.8554 even as its recall improves. Its thresholded decisions get better while its ranking of the small number of low-confidence normal flows gets worse; the other three models move by less than 0.001 on ROC-AUC. This seems to indicate that whatever separates Normal from Attack is

not in the near-duplicate structure of the dataset and that the models are not relying on having seen a near-identical flow before to perform.

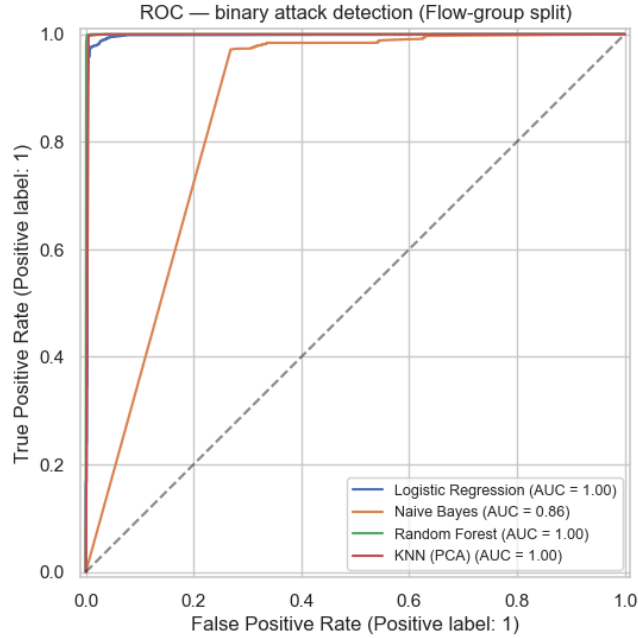


Figure 6. ROC curves for binary attack detection, flow-group split.

3.2 Multi-Class Classification

As anticipated, the multi-class attribution is harder, and unlike the binary task, it is definitely sensitive to the splitting strategy (Table 4). Random Forest is again the best model, reaching a macro-F1 0.9862 and a macro-recall 0.9772 on the flow-group split, with KNN in the PCA feature space second at a 0.9736 macro-F1.

Model	Macro-F1 (Random)	Macro-F1 (Flow-group)	Macro-recall (Flow-group)	Weighted-F1 (Flow-group)	Accuracy (Flow-group)
Random Forest	0.9901	0.9862	0.9772	0.9986	0.9986
KNN (PCA)	0.9698	0.9736	0.9697	0.9966	0.9966
Logistic Regression	0.8305	0.8196	0.9639	0.9898	0.9887
Naive Bayes	0.5903	0.5902	0.8083	0.9220	0.9152

Table 4. Multi-class results under both splits. Macro-F1 is the primary metric.

Two results are worth pulling out of Table 4. First, weighted-F1 barely moves between the two splits. This gives credence to exactly why it is not a valuable metric here. Just like the data itself, the metric is dominated by DOS_SYN_Hping, which is classified perfectly under both splits on 26,521 test flows. Second, Naive Bayes turns out to be unusable for this task with a macro-F1 score of 0.59. This is most likely due to its conditional independence assumption being contradicted by the 52 highly correlated feature pairs. This could be worked around, but it's here mostly for a comparison, not the main model. Logistic Regression reached a 0.9639 macro-recall but only a 0.8196 macro-F1, meaning class weighting biases rare-class recall by over-predicting those classes, and macro-F1 penalizes the resulting false positives. The confusion matrix of the Random Forest model on the flow-group split is shown in Figure 7.

Confusion matrix — Random Forest, Flow-group split

Actual	ARP_poisoning	1775	0	0	0	0	0	0	0	2	0	10	0
	DDOS_Slowloris	2	108	0	0	0	0	0	0	0	0	0	0
	DOS_SYN_Hping	0	0	26521	0	0	0	0	0	0	0	0	0
	MQTT_Publish	0	0	0	1047	0	0	0	0	0	0	0	0
	Metasploit_Brute_Force_SSH	2	0	0	0	7	0	0	0	0	0	0	0
	NMAP_FIN_SCAN	0	0	0	0	0	6	0	0	0	0	0	0
	NMAP_OS_DETECTION	0	0	0	0	0	0	578	0	0	0	0	0
	NMAP_TCP_scan	2	0	0	0	0	0	0	264	0	0	0	0
	NMAP_UDP_SCAN	3	0	0	0	0	0	0	0	723	1	1	0
	NMAP_XMAS_TREE_SCAN	0	0	0	0	0	0	0	0	0	464	0	0
	Thing_Speak	21	0	0	0	0	0	0	0	2	0	1881	0
	Wipro_bulb	0	0	0	0	0	0	0	0	0	0	0	48
		ARP_poisoning	DDOS_Slowloris	DOS_SYN_Hping	MQTT_Publish	Metasploit_Brute_Force_SSH	NMAP_FIN_SCAN	NMAP_OS_DETECTION	NMAP_TCP_scan	NMAP_UDP_SCAN	NMAP_XMAS_TREE_SCAN	Thing_Speak	Wipro_bulb
		Predicted											

Figure 7. Multi-class confusion matrix, Random Forest, flow-group split.

Table 5 shows where the drop between splits actually comes from. Every class moves by less than 0.04 except one, and three are unchanged entirely.

Class	Train flows	Test flows	Random	Flow-group	Difference
Metasploit_Brute_Force_SSH	27	9	1.0000	0.7778	-0.2222
NMAP_TCP_scan	736	266	1.0000	0.9925	-0.0075
DDOS_Slowloris	423	110	0.9850	0.9818	-0.0031
Thing_Speak	5,750	1,904	0.9901	0.9879	-0.0022
ARP_poisoning	5,838	1,787	0.9948	0.9933	-0.0015
DOS_SYN_Hping	63,568	26,521	1.0000	1.0000	0.0000
NMAP_FIN_SCAN	22	6	1.0000	1.0000	0.0000
NMAP_OS_DETECTION	1,422	578	1.0000	1.0000	0.0000
NMAP_UDP_SCAN	1,856	728	0.9923	0.9931	0.0009
MQTT_Publish	3,095	1,047	0.9990	1.0000	0.0010
NMAP_XMAS_TREE_SCAN	1,546	464	0.9980	1.0000	0.0020
Wipro_bulb	171	48	0.9636	1.0000	0.0364

Table 5. Per-class recall, Random Forest, by splitting strategy. Sorted by change.

The only class that falls meaningfully is Metasploit_Brute_Force_SSH, one of the two classes with the fewest independent examples. It is recalled perfectly under the random split, but under the flow-group split the model misclassifies 2 of its 9 examples. NMAP_FIN_SCAN, which is just as rare with 6 test flows, is recalled perfectly under both. This contrast is a clear warning about the numbers themselves. At six and nine test flows, a class can look solved or unsolved depending on which flows ended up in the test set.

3.3 KNN and the Choice of Distance Space

Table 6 measures what the PCA-reduced space actually produces. Accuracy is mostly a wash with 0.4 macro-F1 points being well inside the margin for error and noise. Importantly though, the reduced version runs in about 40% less time on a third of the dimensions.

Variant	Macro-F1	Binary recall	Dimensions	Fit + predict
KNN, all 94 features	0.9700	0.9990	94	1.6 s
KNN, PCA (95% variance)	0.9736	0.9991	30	1.0 s

Table 6. KNN in the full feature space versus the PCA-reduced space, flow-group split.

The conclusion is that PCA makes KNN cheaper but not really better here. The redundancy argument is valid in general, but it is not what limits KNN on this dataset. What the reduced space offers is a distance function that is not double or triple counting flow durations, needs less computation, and a model whose behavior can be described in terms of 30 orthogonal directions rather than 94 entangled ones.

3.4 Best Model Analysis

Random Forest wins on both tasks and under both splits, so its underlying feature importances are worth exploring (Figure 8). However, there are two things to keep in mind before looking at these results. First, the importance scores are very spread out, with the highest-scoring feature getting only a 0.045. This happens because highly similar features end up sharing the credit with each other. As a result, the ranking just shows us which general groups of features matter most, rather than pointing to a single deciding column. Second, the math used to calculate these scores naturally gives an advantage to columns that contain many unique values. This built-in bias is worth keeping in mind for a feature like `id.resp_p`, which still lands in the top ten despite being a single port number.

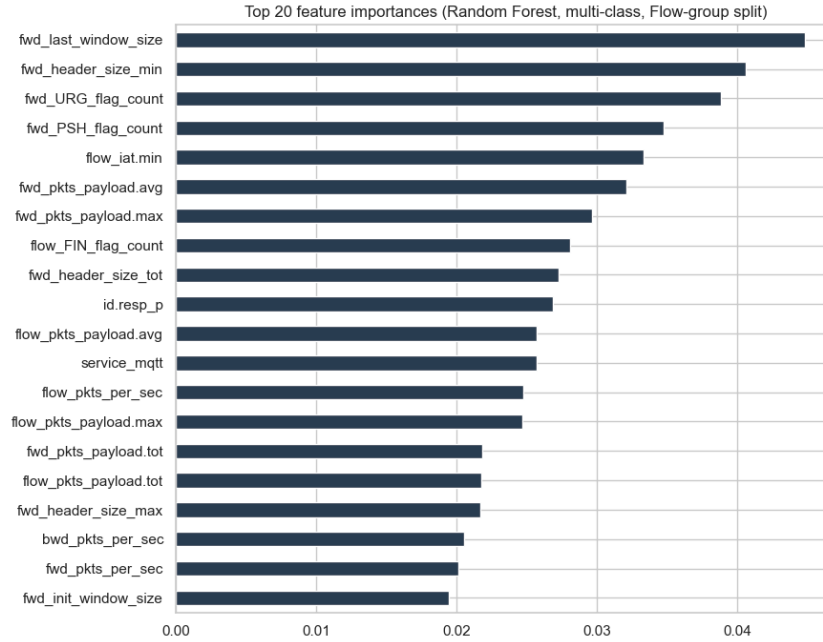


Figure 8. Top 20 Random Forest feature importances, multi-class, flow-group split.

Even keeping those warnings in mind, the most important features still make logical sense together. The top indicators include window and header sizes, counts of specific network flags (like URG, PSH, and FIN), payload statistics, the destination port, and the MQTT service. In simple terms, these details act like a digital fingerprint that reveals exactly what kind of tool or attack created the network traffic.

4. Discussion

The most useful and meaningful insights from this project came from not trusting initial good results. The binary scores were high enough to arouse suspicion, and investigating why produced the flow-group analysis that shaped the project. The results came out mixed, which is itself worth reporting. The binary results held up well and survived. It holds under a split where no near-identical flow appears on both sides, so its high performance is not an artifact of duplicated traffic. On the other hand, the multi-class classification of the rare classes proved less stable, though that instability is concentrated in a single class.

The unexpected challenge was that the obvious fix did not work. Dropping the 5,195 exact duplicate rows is the standard response to duplicated data, and it removed the exact matches. Regardless, 86% of test flows still sat within 0.01 of a training flow afterward. Evidently, the problem was near-duplication rather than exact duplication. Correcting duplication alone wasn't enough, which is why the grouped split was necessary.

The second challenge was dealing with unreliable results for the rarest categories in the data. NMAP_FIN_SCAN contributes 6 test flows, and Metasploit_Brute_Force_SSH contributes 9, so one misclassification can move recall by more than 10 points. Under the flow-group split the first is recalled perfectly, and the second loses 22 points, even though they are almost equally rare. Because of this, the specific scores for these rare attacks shouldn't be treated as exact measurements but rather as rough estimates. What we can trust, however, is the overall trend. The instability shows up only in the smallest classes, which is where it is most expected. Several limitations bound what this analysis can claim:

- The data is synthetically generated, so it is difficult to gauge how well the models would generalize to the real world without more in-depth domain knowledge and real-world testing.
- An unfortunate reality is that the models may be partially recognizing an underlying pattern of which tools/processes generated a flow rather than detecting intrusion in a broader sense.
- The rare-class estimates rest on 6 to 9 test flows, which is too few to support a precise number.
- Each split was evaluated once. Cross-validation could bolster the results further and give more in-depth metrics.

- No extensive hyperparameter tuning was performed.

5. Conclusion and Recommendations

This project set out to detect and classify intrusions in IoT network traffic, and the results are strong on one goal and somewhat qualified on the other. Binary classification exceeded the initial proposed $\geq 95\%$ recall target with room to spare. Random Forest proved practically perfect at discerning normal from attack network activity, with 0.9997 recall and a 0.9996 ROC-AUC. Furthermore, these results held under both splitting strategies, providing strong evidence that the near-perfect score does not depend on duplicated traffic but rather that the patterns are separated enough to be classified well.

Multi-class classification posed the larger challenge, and the scarcity of the rare classes was the limiting factor. Random Forest reached a macro-F1 0.9862 and classified nine of twelve classes at recall of 0.99 or better, with the residual error concentrated in the rarest classes. That residual error moves around between splits. NMAP_FIN_SCAN and Metasploit_Brute_Force_SSH are almost equally rare, yet under the flow-group split one is recalled perfectly and the other loses 22 points. Underneath both results sits the reality of the dataset itself. It contains roughly 18,954 distinct flows rather than 123,117 independent ones, and how the train/test boundary is drawn regarding that structure moves the rare-class results while leaving the binary results untouched.

Five recommendations come from this project. First, replace the single flow-group split with a repeated group-aware cross-validation. This would allow some sort of an error bound to be put on rare-class numbers that single-fold estimates on 6 and 9 flows cannot support. Second, apply SMOTE or targeted oversampling inside training folds, giving the two smallest classes a better chance without potentially reintroducing leakage. Third, it would be interesting to run a

leave-one-attack-type-out evaluation, withholding an entire attack class from training and testing whether the binary detector still flags it. That would establish whether this is intrusion detection or tool/process recognition. Fourth, for any deployment, it would be necessary to tune the decision threshold against real alert-handling capacity rather than defaulting to 0.5. Fifth and finally, it would be interesting to see how efficient a model could be fashioned for real world application. In large and high-flow infrastructure, a tool that can run quickly and at scale would be of paramount importance.

More broadly, this project serves as a reminder that a machine learning model is only as good as the data it learns from. Decisions about data splitting and handling imperfections are just as foundational to success as algorithm selection and hyperparameter tuning, and they must be approached with the same care and scrutiny.

AI Use Disclosure

Claude (Anthropic) was used for general debugging and troubleshooting. It was also used in a collaborative manner to provide suggestions and aid in building the scikit-learn pipeline and helper functions present in Section 1.1 of the notebook.

References

- Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 100804.
<https://doi.org/10.1016/j.patter.2023.100804>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Sharmila, B. S., & Nagapadma, R. (2023). RT-IoT2022 [Data set]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5P338>